

Documentație

Titlu

Aethelgard: Rise of the Fallen Crown

Gen

Joc 2D tile-based, top-down adventure

Poveste

Jucătorul controlează un cavaler trimis să recupereze coroana pierdută a regatului Aethelgard. Regatul a fost corupt de o forță întunecată, iar eroul trebuie să treacă prin trei zone periculoase, să evite sau să înfrunte inamici, să găsească obiecte-cheie și să ajungă la confruntarea finală

Obiectiv

Obiectivul principal al jocului este parcurgerea celor trei niveluri, depășirea obstacolelor și ajungerea la confruntarea finală, în urma căreia jucătorul recuperează coroana regatului.

Mecanici

- deplasare cu tastatura
- coliziuni cu pereți / obstacole
- colectare iteme
- interacțiuni cu NPC
- inamici simpli
- scor pe baza itemelor, timpului și progresului
- save/load
- trecere între niveluri

Nivelul 1 — The Cursed Forest

- hartă exterioară
- copaci, pietre, drumuri blocate
- inamici simpli
- obiectiv: găsește cheia ruinelor

Nivelul 2 — Beneath the Castle

- hartă interioară
- coridoare, capcane, uși
- NPC / spirit guardian
- obiectiv: găsește artefactul care deschide sala finală

Nivelul 3 — The Great Hall

- nivel final
- confruntare cu antagonistul
- obiectiv: recuperează coroana și finalizează jocul

1.Contextul jocului

Aethelgard: Rise of the Fallen Crown este un joc 2D de tip top-down adventure, bazat pe un sistem tile-based. Acțiunea se desfășoară într-un regat fantasy aflat în ruină, corupt de o forță întunecată. Jucătorul controlează ultimul cavaler loial coroanei și trebuie să exploreze zone periculoase, să depășească obstacole, să interacționeze cu personaje și să recupereze coroana pierdută. Jocul combină explorarea, progresia între niveluri, sistemul de scor și mecanica de salvare/încărcare a progresului.

2. Proiectarea sistemului / gameplay

Jocul începe dintr-un meniu principal, din care utilizatorul poate porni o sesiune nouă sau poate încărca un progres salvat. În timpul jocului, personajul principal se deplasează pe hartă folosind tastatura. Fiecare nivel este construit din tile-uri, care definesc terenul, obstacolele și elementele interactive. Sistemul de joc va actualiza permanent starea personajului, a inamicilor și a hărții printr-o buclă principală de joc (main loop), iar modificările vor fi afișate pe ecran prin randare grafică. Progresul se realizează prin explorare, găsirea unor obiecte-cheie, evitarea sau confruntarea inamicilor și trecerea la nivelul următor. Jocul se încheie atunci când jucătorul ajunge în zona finală și recuperează coroana sau înfruntă antagonistul principal.

3. Șabloane de proiectare utilizate

Singleton

Pattern-ul Singleton va fi utilizat pentru gestionarea resurselor globale ale jocului, cum ar fi încărcarea asseturilor grafice sau conexiunea la baza de date. Astfel, aplicația va avea o singură instanță responsabilă de aceste resurse, evitând duplicarea și consumul inutil de memorie.

Factory

Pattern-ul Factory va fi utilizat pentru crearea obiectelor de tip Enemy. În loc să instanțiem direct inamicii în cod, logica de creare va fi centralizată într-o clasă specializată, numită EnemyFactory, ceea ce face proiectul mai ușor de extins și întreținut.

State (opțional)

Pattern-ul State poate fi utilizat ulterior pentru gestionarea stărilor jocului, precum Main Menu, Playing, Pause, Game Over și Victory. În etapa actuală, acest pattern este prevăzut ca posibilă extindere a arhitecturii jocului.

4. Proiectarea interfeței cu utilizatorul

Interfața jocului va include un meniu principal, ecranul de joc propriu-zis, un HUD minimal și ecrane finale de tip Game Over / Victory. În meniul principal, utilizatorul va putea începe un joc nou sau încărca un progres existent. În timpul jocului, HUD-ul va afișa informații esențiale precum scorul, progresul, nivelul curent și eventual starea personajului. Interfața va fi simplă, clară și adaptată tematicii fantasy întunecate, folosind culori închise și elemente vizuale sugestive.



Figura 1. Mockup pentru meniul principal al jocului

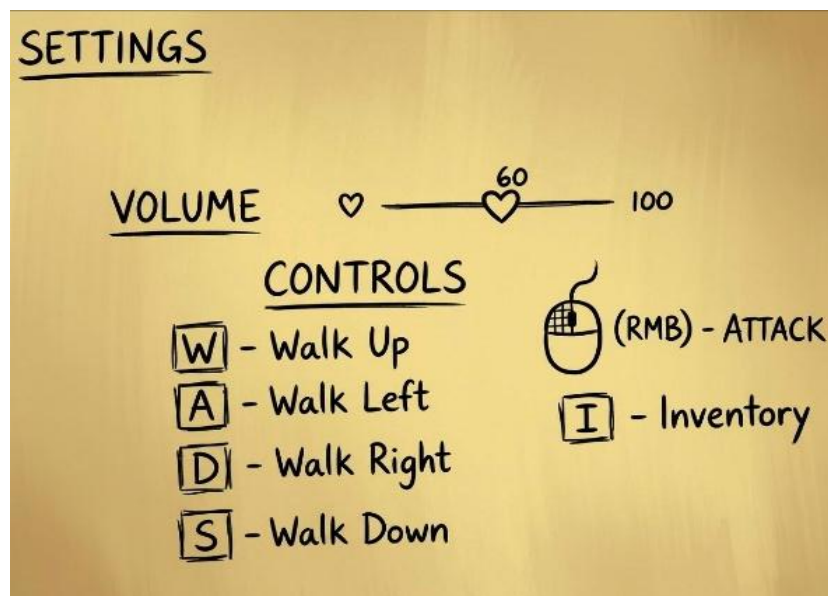


Figura 2. Mockup pentru ecranul de setări și controale

Pentru susținerea proiectării interfeței, au fost realizate mockup-uri conceptuale pentru meniul principal și pentru ecranul de setări/controale ilustrate în figurile de mai sus.

5. Proiectarea conținutului

Conținutul jocului este construit în jurul unei lumi dark fantasy și al unui parcurs narativ împărțit în trei capitole. Personajul principal este ultimul cavaler loial coroanei regatului Aethelgard, trimis să recupereze coroana pierdută și să restabilească ordinea într-un regat corupt de magie întunecată. Antagonistul principal este Malakar, un vrăjitor care a adus regatul în ruină și care reprezintă obstacolul final al poveștii. Structura capitolelor și confruntarea finală cu Malakar apar și în materialele auxiliare, ceea ce face alegerea foarte potrivită pentru proiect.

În joc vor exista mai multe tipuri de entități interactive, modelate printr-o superclasă comună Entity. Personajul principal (Player) va fi controlat de utilizator și va putea explora

hărțile, colecta obiecte și interacționa cu alte personaje. Inamicii (Enemy) vor fi de tipul Corrupted Guard și Shadow Beast, iar rolul lor va fi să blocheze progresul jucătorului și să introducă dificultate pe parcursul nivelurilor. În plus, va exista cel puțin un NPC important (NPC), de exemplu Spirit Guardian sau Imprisoned Wizard, care va oferi informații, va ghida jucătorul sau va influența progresul poveștii.

Obiectele principale ale jocului vor include cheia ruinelor, artefactul necesar pentru accesul în zona finală, coroana pierdută și obiecte consumabile precum potion. Aceste iteme vor avea rol funcțional în progresia dintre niveluri și pot contribui la sistemul de scor și la deblocarea unor căi alternative. Pentru a face jocul mai interesant, conținutul va include și un sistem de progresie bazat pe XP și points, precum și realizări speciale de tip achievements și endings. Exemple potrivite sunt: *First Blood*, *Into the Woods*, *The Forgotten Cell*, *Master of Mercy*, precum și ending-urile *Crown Reclaimed* sau *Exile of Malakar*.

6. Proiectarea nivelurilor

Jocul va conține minim trei niveluri, conform cerinței proiectului. Fiecare nivel va avea o hartă proprie, elemente de decor, obstacole, inamici și un obiectiv clar care permite progresul către etapa următoare. Hărțile vor fi concepute astfel încât să reflecte atât povestea, cât și creșterea dificultății. Pentru proiectarea efectivă a hărților se poate utiliza Tiled Map Editor, așa cum este recomandat în cerință.

Nivelul 1 — The Cursed Forest

Primul nivel reprezintă pădurea blestemată de la marginea regatului și are rol de introducere în atmosferă și mecanici. Harta va include zone împădurite, pietre, obstacole naturale, trasee blocate și primele confruntări cu inamici. Obiectivul principal al nivelului este găsirea cheii ruinelor și deschiderea căii către următoarea zonă. În acest nivel pot apărea primele alegeri, descoperiri ascunse și primul achievement.



Figura 3. Schiță conceptuală pentru nivelul 1

Figura de mai sus prezintă o schiță conceptuală a primului nivel, incluzând traseul principal, plasarea unor inamici și existența unei rute ascunse.

Nivelul 2 — Beneath the Castle

Al doilea nivel are loc în subteranele castelului și introduce o atmosferă mai tensionată, cu coridoare, capcane, uși și camere ascunse. Aici jucătorul întâlnește NPC-ul principal, de exemplu vrăjitorul întemnițat, și descoperă adevăruri importante despre regat și despre Malakar. Obiectivul nivelului este obținerea artefactului care deschide sala finală. Acest nivel poate conține și o rută secretă sau o decizie care influențează ending-ul.

Nivelul 3 — The Great Hall

Al treilea nivel este zona finală a jocului și include apropierea de sala tronului, confruntarea cu forțele lui Malakar și decizia finală. Harta poate conține o arie centrală mare, inamici mai puternici și confruntarea cu antagonistul principal. Obiectivul final este recuperarea coroanei și deblocarea unui ending, bun, rău sau secret, în funcție de progresul și alegerile jucătorului.

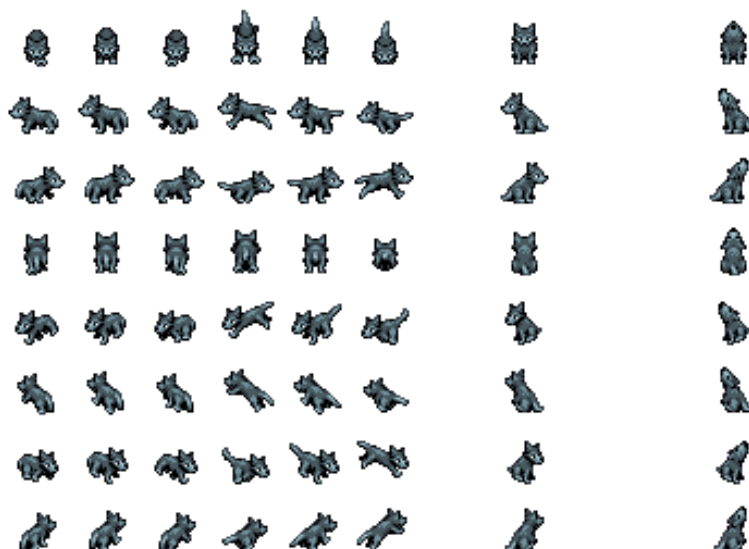
7. Sprite-uri și resurse grafice

Proiectul va utiliza minimum două game sprite-uri, conform cerinței. Primul sprite va reprezenta personajul principal, adică cavalerul controlat de utilizator, iar al doilea va reprezenta un inamic al jocului, de exemplu un lup ostil (Wolf Enemy). În plus, vor fi utilizate tile-uri pentru teren și decor, precum grass, stone, wall, water și dungeon floor. Resursele grafice pot fi obținute din biblioteci specializate de asset-uri sau realizate/adaptate în editoare dedicate precum Aseprite, GIMP sau alte unelte recomandate în suportul de curs.

Sprite 1: Knight Player Sprite



Sprite 2: Wolf Enemy Sprite



8. Sistem de progresie, scor și bonusuri

Jocul va include un sistem de progresie bazat pe XP și points. XP va fi utilizat pentru nivelul jucătorului, iar points vor reprezenta scorul total și vor fi folosite pentru clasamentul celor mai buni jucători. O variantă de progresie potrivită este: Level 1 — 0 XP, Level 2 — 100 XP, Level 3 — 250 XP, Level 4 — 450 XP, Level 5 — 700 XP. Sursele de XP și points pot include finalizarea capitolelor, înfrângerea inamicilor, descoperirea unor zone ascunse și deblocarea ending-urilor.

9. Salvarea progresului, baza de date și tratarea excepțiilor

Jocul va permite salvarea și încărcarea progresului dintr-o bază de date, conform cerinței proiectului. Vor fi salvate cel puțin nivelul curent, poziția jucătorului, progresul, XP-ul, points-urile și scorul obținut. De asemenea, se va ține evidența celor mai bune scoruri și a celor mai buni jucători. Pentru tratarea excepțiilor, vor fi definite clase specifice, de exemplu: `InvalidSaveException`, `ResourceLoadException`, `DatabaseConnectionException`, `InvalidLevelException` și `SpriteNotFoundException`. Acestea vor ajuta la gestionarea robustă a erorilor în timpul execuției, în special la încărcarea resurselor, salvarea progresului și comutarea între niveluri.

10. Sarcinile săptămânale ale membrilor echipei

Planificarea sarcinilor a fost realizată în conformitate cu cerințele oficiale ale proiectului. Task-urile vor fi urmărite în GitHub Projects și corelate cu commit-urile din repository. Începând cu săptămâna 8, fiecare membru al echipei va realiza cel puțin un commit pe săptămână, conform împărțirii sarcinilor.

Notă: Planificarea de mai jos urmează calendarul oficial al proiectului. Activitățile de preproducție desfășurate în săptămâna 7 sunt centralizate în documentația predată la începutul săptămânii 8.

Săptămâna 7 — Etapa de preproducție (predare la începutul săptămânii 8)

- **Membrul 1(Grigoraș Ioana-Daria):** definirea conceptului jocului, redactarea contextului, a gameplay-ului, a conținutului și a proiectării nivelurilor.
- **Membrul 2(Popescu Mircea):** realizarea diagramei UML, alegerea sprite-urilor și a resurselor grafice, descrierea interfeței cu utilizatorul, completarea bibliografiei și verificarea finală a documentului PDF.
- **Rezultat urmărit:** documentație completă, exportată în PDF și pregătită pentru încărcare la începutul săptămânii 8.

Săptămâna 8 — Meniul principal și nivelul 1

- **Membrul 1(Grigoraș Ioana-Daria):** implementarea meniului principal și a navigării de bază.
- **Membrul 2(Popescu Mircea):** încărcarea hărții pentru nivelul 1 și configurarea primelor tile-uri.
- **Rezultat urmărit:** meniu funcțional și nivelul 1 încărcat.

Săptămâna 9 — Deplasarea personajului și camera

- **Membrul 1(Grigoraș Ioana-Daria):** implementarea deplasării personajului principal pe hartă.

- **Membrul 2(Popescu Mircea):** implementarea camerei și adaptarea afișării în funcție de poziția jucătorului.
- **Rezultat urmărit:** personaj controlabil și cameră funcțională.

Săptămâna 10 — Nivelurile 2 și 3, inamici și NPC

- **Membrul 1(Grigoraș Ioana-Daria):** adăugarea nivelurilor 2 și 3, inclusiv obiectivele de progresie dintre niveluri.
- **Membrul 2(Popescu Mircea):** implementarea inamicilor și a NPC-urilor, precum și a interacțiunilor de bază cu acestea.
- **Rezultat urmărit:** toate cele trei niveluri disponibile, cu entități interactive.

Săptămâna 11 — Coliziuni

- **Membrul 1(Grigoraș Ioana-Daria):** implementarea coliziunilor cu pereți, obstacole și zone blocate.
- **Membrul 2(Popescu Mircea):** testarea și reglarea coliziunilor pentru player, inamici și obiecte interactive.
- **Rezultat urmărit:** mișcare corectă și blocarea traversării elementelor solide.

Săptămâna 12 — SAVE / LOAD și baza de date

- **Membrul 1(Grigoraș Ioana-Daria):** implementarea sistemului de salvare și încărcare a progresului.
- **Membrul 2(Popescu Mircea):** integrarea bazei de date pentru progres, scoruri și best players.
- **Rezultat urmărit:** jocul poate salva și încărca progresul din baza de date.

Săptămâna 13 — Polish, patterns, excepții și bonusuri

- **Membrul 1(Grigoraș Ioana-Daria):** verificarea și integrarea șabloanelor de proiectare în implementare, tratarea excepțiilor și curățarea codului.
- **Membrul 2(Popescu Mircea):** finisarea interfeței, adăugarea de NPC suplimentari dacă este cazul și implementarea unuia sau mai multor bonusuri, precum fog of war, achievements sau ending secret.
- **Rezultat urmărit:** joc stabil, coerent și pregătit pentru prezentare.

Săptămâna 14 — Prezentarea finală

- **Membrul 1(Grigoraș Ioana-Daria):** pregătirea demonstrației tehnice și prezentarea implementării.
- **Membrul 2(Popescu Mircea):** pregătirea explicațiilor despre design, niveluri, UI, bază de date și bonusuri.
- **Rezultat urmărit:** prezentare finală completă a proiectului.

11. Explicația diagramei UML

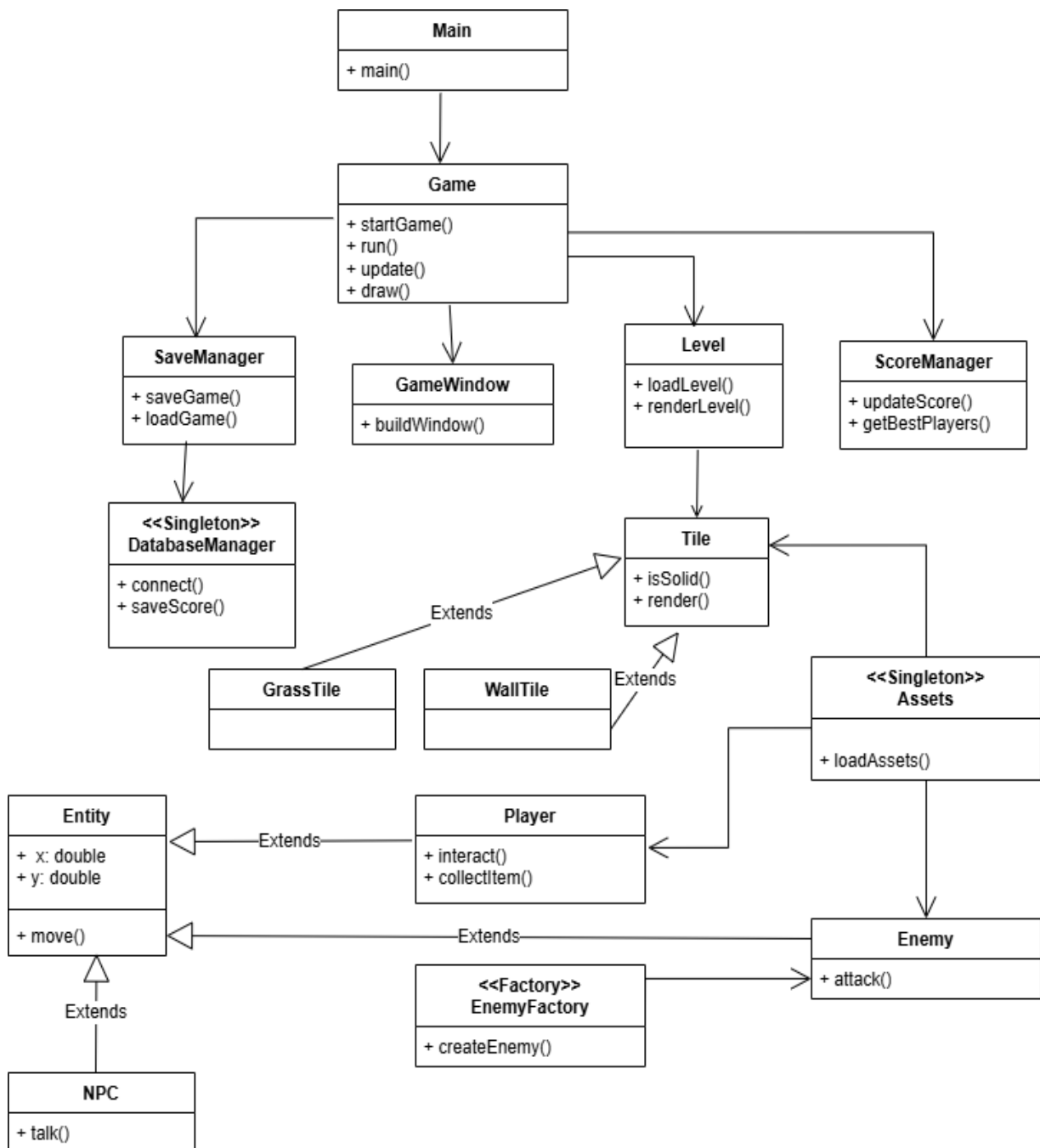


Diagrama UML de clase prezintă structura preliminară a jocului *Aethelgard: Rise of the Fallen Crown*. Clasa **Main** reprezintă punctul de intrare în aplicație și creează instanța clasei **Game**, care gestionează bucla principală a jocului prin metodele `startGame()`, `run()`, `update()` și `draw()`. Clasa **Game** colaborează cu **GameWindow**, **Level**, **ScoreManager** și **SaveManager**, coordonând logica principală a aplicației. Structura hărții este construită pe baza clasei **Tile**, din care derivă subclase precum **GrassTile** și **WallTile**. Entitățile dinamice ale jocului sunt modelate prin superclasa **Entity**, din care derivă **Player**, **Enemy** și **NPC**. Pentru gestionarea resurselor grafice și a conexiunii la baza de date sunt utilizate elemente de tip Singleton, prin clasele **Assets** și **DatabaseManager**, iar pentru crearea inamicilor este utilizat pattern-ul Factory, prin clasa **EnemyFactory**.

12. Bibliografie

- [1] R. Nystrom, *Game Programming Patterns*.
- [2] OpenGameArt, “Game Art Assets.”
- [3] Tiled Map Editor, “Official Website.”
- [4] diagrams.net, “draw.io.”
- [5] Aseprite, “Pixel Art Tool.”
- [6] GIMP, “GNU Image Manipulation Program.”

Resursele pentru asset-uri și editoare: OpenGameArt, PixelProspector, Aseprite, GIMP, Inkscape